umyo_testing.py

1. pbt_init() → display_stuff.py

plot_emg. array with 2000 zeros → plot_emg becomes global
plot_spg array with 800 zeros
plot_ax array with 200 zeros
plot_ay array with 200 zeros
plot_az array with 200 zeros
plot_q array with 800 zeros

and max_devices becomes global ~~where~~ which equals 64

2. umyo_parse_preprocessor(data) → umyo_parser.py

umyo_parse_preprocessor() function returns cht but it makes changes of variables such as parse_buf etc.

parse_buf, ~~append~~ atend (data)
functionality described in the guide to this map. In fact this function makes the parsing of data from umyo and returns the count of unparsed data

3. umyo_parse() function makes parsing of data from umyo by umyo_protocol which looks like this, it doesn't return anything

pp=pos
rssi=parse_buf[pp-1] ⟷ the value pp-1 will be at first 2 like in the preprocessor function

Thus the structure of packet looks like this:

79, 213, RSSI, packet_id, packet_len, unit_id, unit_id, unit_id, unit_id, ~~useful payload~~ packet_type

in the umy_parse function we also fill up our umyo_list[idx] I mean our devices' classes and we fill these things.

umyo_list[idx].data_count = packet_type - 80
umyo_list[idx].packet_type = 80
umyo_list[idx].rssi = rssi
umyo_list[idx].batt = 2000 + pb1 · 10
umyo_list[idx].version = pb2
umyo_list[idx].data_array[x] = val
umyo_list[idx].device_spectr[x] = val

umyo_list[idx].Qsg[0] = qww
umyo_list[idx].Qsg[1] = qwx
umyo_list[idx].Qsg[2] = qwy
umyo_list[idx].Qsg[3] = qwz
umyo_list[idx].yaw = yaw → ~~[illegible]~~ radyan cevirme
umyo_list[idx].pitch = umyo_list[idx].pitch * 0,95 + 0,05 · pitch

if (pitch > 2000 and umyo_list[idx].pitch < -2000):
    umyo_list[idx].pitch = pitch
if (pitch < -2000 and umyo_list[idx].pitch > 2000):
    umyo_list[idx].pitch = pitch
umyo_list[idx].roll = roll → [illegible]

4. id2idx() function makes changes of such variables as unseencnt, umyolist, but unseen variable can be changed I mean deleted

So, in fact it returns id of current device it's working with, but first two times it returns count of umyo devices in umyo_list. And it also adds ~~data~~ umyo to umyo_list

5. umyo_class: uMyo(uid) creates a class of uMyo ~~with~~ that looks like this. At first

self.last_pack_id = 0
self.unit_id = uid
self.packet_type = 0
self.data_count = 0
self.batt = 0
self.version = 0
self.steps = 0
self.data_id = 0
self.prev_data_id = 0
self.data_array = [0]·64 # in case of further changes, now 16 data points
self.device_spectr = [0]·16
self.Qsg = [0, 0, 0, 0]
self.zeroQ = [0, 0, 0, 0]
self.yaw = 0
self.pitch = 0

```
umya_list[idx].ax=ax
umya_list[idx].ay=ay
umya_list[idx].az=az
umya_list[idx].mag_angle=mag_angle
```

```
self.roll=0
self.dev_yaw=0
self.dev_roll=0
self.update_time=0
self.yaw_speed=0
self.pitch_speed=0
self.roll_speed=0
self.mag_angle=0
```

The structure of Jorton

[70, 213, RSSI, Packet_id, packet_len, unit_id, unit_id, unit_id, unit_id, packet_type, power_id, pb1, pb2, pb3, data_id, hb, lb, hb, lb... until pp=32 or other value, hb,

lb... until pp=40, and with is hb, ut lb, hb, lb, hb, lb, hb, lb, hb, lb, hb, lb, hb, lb, lb, lb, hb, lb, hb, lb, hb, lb, hb, lb, hb, lb, hb, lb, hb, lb, until pp=32 or other value, hb,

IMU (Intertial Measurement Unit) data

Data EMG

Magnetic data

mx, my, mz

r:_quat_math.py

ryr= quat_math.sV(0,1,0) where sV is just a tuple with a name X=0, Y=1, Z=0 | 3D Vector

Qsg=quat_math.sQ(q.ww, q.wx, q.wy, q.wz) | Quantenion data

nyr= quat_math.rotate_V(Qsg, nyr) # makes a rotation of 3D vector that happens like this $v' = q \cdot v \cdot q^*$ where q^* is a quaternion conjugate (complex conjugate) which is defined like that sQ(q.w, -q.x, -q.y, -q.z)

def q_mult(q1: sQ, q2: sQ) → sQ:

$w = q1.w \cdot q2.w - (q1.x \cdot q2.x + q1.y \cdot q2.y + q1.z \cdot q2.z)$

$x = q1.w \cdot q2.x + q2.w \cdot q1.x + q1.y \cdot q2.z - q1.z \cdot q2.y$

$y = q1.w \cdot q2.y + q2.w \cdot q1.y + q1.z \cdot q2.x - q1.x \cdot q2.z$

$z = q1.w \cdot q2.z + q2.w \cdot q1.z + q1.x \cdot q2.y - q1.y \cdot q2.x$

M= sV(mx, my, mz)

M= v_renorm(M)

```
r= √(v.x·v.x+v.y·v.y+v.z·v.z)
x=0
y=0
z=0
if (r>0):
    m=1/r
    x=v.x·m
    y=v.y·m
    z=v.z·m
return sV(x,y,z)
```

A= sV(ax, ay, -az) Here az is negative because in almost all IMU-sensors axis z is directed up to ~~the~~ from the divice but gravity directed to the down

A= v_renorm(A)

if M close to A

m_vent= v_dot(A,M) → $(A_x \cdot M_x + A_y \cdot M_y + A_z \cdot M_z)$ → proection of magnetic vector to the magnetic direction

M_hor= sV(M.x - m_vent·A.x, M.y - m_vent·A.y, M.z - m_vent·A.z) we remove from general magnetic field its vertical component, leaving only horizontal

M_hor= v_renorm(M_hor)

H= sV(0, 1, 0) → we work with it as the direction forward not to make addiction of vertical position to our poluses.

h_vent= v_dot(A,H) → $(A_x \cdot H_x + A_y \cdot H_y + A_z \cdot H_z)$

H_hor= sV(H.x - h_vent·A.x, H.y - h_vent·A.y, H.z - h_vent·A.z)

90° A

then v_dot equals 1 if 0 then its 90° between them, So m_vert is the level of m's verticality

H_hor = v_renorm(H_hor) → logically that's normalization of vector. Normalization guarantees that we work with directions, not with absolute values

HM = quat_math.v_mult(H_hor, M_hor)

→ $H_hor_y \cdot M_hor_z - H_hor_z \cdot M_hor_y \Rightarrow x$

$H_hor_z \cdot M_hor_x - H_hor_x \cdot M_hor_z \Rightarrow y$

$H_hor_x \cdot M_hor_y - H_hor_y \cdot M_hor_x \Rightarrow z$

asign = -1

if (quat_math.v_dot(HM, A) < 0):

asign = 1

mag = quat_math.v_dot(H_hor, M_hor)

mag_angle = asign · math.acos(mag)

pitch = round(math.atan2(az, ax) · 1000)

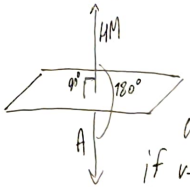
umyo_list[idx].@sg[0, 1...] = gyro, gyro, ...

umyo_list[idx].pitch = umyo_list[idx].pitch · 0,95 + 0,05 · pitch

this change makes smoothing of pitch 95% of old + 5% of new

then goes the processing of the case when our changes are too sharp

1



$|a| \cdot |b| \cdot \cos\alpha \Rightarrow$ the condition works like this if v_dot(HM, A) < 0

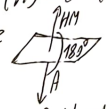
that means that the angle between them is higher than 90° which means they're opposite

but the very HM is 90° to the horizontal space because H_hor. M_hor equals a vector which is 90° to the horizontal space

But then we have to define where exactly HM will be down or up. It's defined by the rule of right hand

2

when α between HM and A is 180 then quat_math.v_dot(HM, A) will be negative cause $\cos(\pi) = \cos(180) = -1$. That means when we see



that we multiply 1 · arccos(mag)

but if that's not true for example

then scalar dot will be 0 and cos(0) = 1 and asign will be -1

5. display_stuff.py

plot_prepare(umyo_parser.umyo.get_list())

this function gets data from our device from umyo_class and makes appends of the global functions in display_stuff.py such as

plot_spg ← spectr data described devices[d].device.spectr[n]

plot_emg ← emg data plot_emg[d].append(val) where val = devices[d].data_array[x]

y_zero ← some part of emg 0.997 · y_zero[d] + 0.003 · val

plot_ax[d].append device.ax

plot_ay

plot_az

plot_a[d].append(devices[d].@sg[0] [1] [2])

H_hor is direction of our device

rule of right hand described like fingers to the direction H_hor ↑ of the right hand. Then fingers directed to the North and the big fingers direction - is the direction of HM

6. display_stuff. plot_cycle_testep

last_data_id[d] = devices[d].data_id

dev_rssi[d] = devices[d].rssi

dev_mag_angle[d] = devices[d].mag_angle

dev_batt[d] = device[d].batt

then we take only last data of plot_emg[d] and plot_spg[d], plot_ax[d], plot_ay[d], plot_az[d]

plot_a,

For plot_emg 2000 last

For plot_spg 800

For plot_ax 200

For plot_ay 200

For plot_az 200

For plot_a[d] 800